

ROS 2 部分详细讲解

校园赛代码中 ROS 2 部分详解（小白版）

目的：把代码里所有 ROS 2 相关的内容讲清楚，并告诉你每个变量/节点/话题/消息类型在代码里哪一行定义。

0. ROS 2 在代码里出现的位置总览

行号	内容	作用
1	<code>#include</code>	引入 ROS 2 C++ 库
14	<code><rclcpp/rclcpp.hpp></code>	
	<code>using namespace</code>	省略 rclcpp:: 前缀
	<code>rclcpp;</code>	
81	<code>class CampusNode :</code>	定义节点类，继承 Node
	<code>public rclcpp::Node</code>	
86	<code>:</code>	给节点起名
	<code>Node("campus_node")</code>	
188	<code>create_subscription<CompressedImage></code>	订阅压缩图像
199	<code>create_subscription<AutoAimFrame></code>	订阅 AutoAimFrame
210	<code>create_publisher<SendData></code>	发布 SendData
711	<code>SendData send_msg;</code>	创建要发的消息
837	<code>image_sub_</code>	图像订阅器成员变量
846	<code>frame_sub_</code>	AutoAimFrame 订阅器成员变量
850	<code>send_data_pub_</code>	SendData 发布者成员变量
854	<code>last_state_</code>	保存自身状态
858	<code>frameCallback(msg)</code>	收到 AutoAimFrame 的回调
882	<code>imageCallback(msg)</code>	收到压缩图像的回调
915	<code>rclcpp::init</code>	初始化 ROS 2
925	<code>rclcpp::spin</code>	让节点持续运行
930	<code>rclcpp::shutdown</code>	关闭 ROS 2

1. 节点类 CampusNode (81-86 行)

```
class CampusNode : public rclcpp::Node // 81
{
public:
    CampusNode() : Node("campus_node") // 85-86
    {
```

```

    ...
}
};

```

- `class CampusNode : public rclcpp::Node`: 定义一个类，继承自 `Node`。继承后 `CampusNode` 自动拥有 `Node` 的所有能力。
- `CampusNode()` : `Node("campus_node")`: 构造函数。后面 : `Node("campus_node")` 是调用父类构造函数，给节点起名字 `campus_node`。
 – 这个名字会显示在 `ros2 node list` 里。

2. 订阅器与发布者成员变量 (837-852 行)

```

rclcpp::Subscription<CompressedImage>::SharedPtr image_sub_; // 839
rclcpp::Subscription<AutoAimFrame>::SharedPtr frame_sub_; // 848
rclcpp::Publisher<SendData>::SharedPtr send_data_pub_; // 852

```

为什么要写成成员变量? 因为创建订阅/发布后，如果不保存到成员变量，函数结束时它们会被销毁，订阅/发布就失效了。成员变量让它们活到节点结束。

- `image_sub_`: 图像订阅器。
- `frame_sub_`: `AutoAimFrame` 订阅器。
- `send_data_pub_`: 瞄准指令发布者。

3. 订阅图像 (187-196 行)

```

image_sub_ = this->create_subscription<sensor_msgs::msg::CompressedImage>(
    "/camera/image_raw/compressed", // 话题名
    10, // 队列长度
    std::bind(&CampusNode::imageCallback, this, std::placeholders::_1)
);

```

- `create_subscription`: 创建订阅者。
- `CompressedImage`: 消息类型，表示压缩图像。
- `"/camera/image_raw/compressed"`: 话题名，游戏端发布的压缩图像话题。
- `10`: 队列长度，最多缓存 10 帧。
- `std::bind(...)`: 绑定回调。

回调签名:

```

void imageCallback(const sensor_msgs::msg::CompressedImage::SharedPtr msg)

```

收到消息后自动调用 `imageCallback`, `msg` 就是图像数据。

4. 订阅 AutoAimFrame (198-207 行)

```
frame_sub_ = this->create_subscription<tdt_interface::msg::AutoAimFrame>(
    "/autoaim/frame",
    1,
    std::bind(&CampusNode::frameCallback, this, std::placeholders::_1)
);
```

- AutoAimFrame 消息包含：图像、相机内参 camera_info、自身状态 state。
 - 话题 /autoaim/frame。
 - 回调 frameCallback：收到后把内参存进 cameraMatrix_/distCoeffs_，把状态存进 last_state_。
-

5. 发布 SendData (209-213 行 + 711-715 行)

创建发布器：

```
send_data_pub_ = this->create_publisher<tdt_interface::msg::SendData>(
    "/target_angles_player_1", 10);
```

发布消息：

```
tdt_interface::msg::SendData send_msg;    // 711
send_msg.yaw = world_yaw;
send_msg.pitch = pitch_deg;
send_msg.if_shoot = true;
send_data_pub_->publish(send_msg);        // 715
```

- SendData 消息有三个字段：yaw/pitch/if_shoot。
 - send_data_pub_->publish(send_msg)：把消息发到话题 /target_angles_player_1。
 - 蓝方 N=1，所以话题是 /target_angles_player_1。
-

6. 两个回调 (856-909 行)

frameCallback (857-879 行)

```
void frameCallback(const AutoAimFrame::SharedPtr msg)
{
    // 从 msg->camera_info.k 填 cameraMatrix_
    // 从 msg->camera_info.d 填 distCoeffs_
    // last_state_ = msg->state;
}
```

为什么要这个回调？ 因为 solvePnP 需要相机内参，而内参在 AutoAimFrame 里。每收到一帧就更新一次内参和自身状态。

imageCallback (881-909 行)

```
void imageCallback(const CompressedImage::SharedPtr msg)
{
    Mat frame = imdecode(msg->data, IMREAD_COLOR);
    if(frame.empty()) { warn; return; }
    processFrame(frame);
}
```

为什么要这个回调? 图像来了才处理。imdecode 把压缩数据解码成 OpenCV 图像。

7. last_state_ (854 行)

```
tdt_interface::msg::AutoAimState last_state_;
```

为什么要它? 我们要把“目标在相机系下的偏角”换成“世界光轴角”。需要知道相机当前的世界 yaw, 而这就存在 last_state_.yaw_degrees 里。

8. main 里的 ROS 2 三件套 (913-933 行)

```
rclcpp::init(argc, argv);           // 初始化 ROS 2
auto node = make_shared<CampusNode>(); // 创建节点
rclcpp::spin(node);                 // 让节点一直跑, 消息来了就回调
rclcpp::shutdown();                 // 关闭 ROS 2
```

为什么 spin 最重要? 没有 spin, 节点创建完马上退出, 收不到任何消息, 也调不到任何回调。

9. 变量/节点/话题对照速查表

名称	类型	定义位置	作用
CampusNode	class	81	节点类
campus_node	string	86	节点名
image_sub_	Subscription::SharedPtr	839	图像订阅器
frame_sub_	Subscription::SharedPtr	848	AutoAimFrame 订阅器
send_data_pub_	Publisher::SharedPtr	852	发布者
last_state_	AutoAimState	854	自身状态
imageCallback	成员函数	882	图像回调
frameCallback	成员函数	858	AutoAimFrame 回调
processFrame	成员函数	218	图像处理主流程
/camera/image_raw/compressed	话题	189	压缩图像话题
/autoaim/frame	话题	200	AutoAimFrame 话题
/target_angles_player_1	话题	211	SendData 话题

名称	类型	定义位置	作用
CompressedImage	消息类型	188	压缩图像
AutoAimFrame	消息类型	199	帧信息 + 内参 + 状态
SendData	消息类型	210	瞄准指令

10. 你要记住的 ROS 2 套路

1. `class XxxNode : public rclcpp::Node` —继承 Node。
2. `XxxNode() : Node("名字")` —构造函数起名。
3. `create_subscription / create_publisher` —订阅/发布。
4. 回调签名: `void cb(const MsgType::SharedPtr msg)`。
5. `std::bind(&XxxNode::cb, this, _1)` —绑定成员函数回调。
6. main 里: `init → make_shared → spin → shutdown`。

本 PDF 与 `main_for_pdf.cpp` 对应。

附录：完整代码

```
#include <rclcpp/rclcpp.hpp>
#include <memory>
#include <functional>
#include <sensor_msgs/msg/compressed_image.hpp>
#include <tdt_interface/msg/auto_aim_frame.hpp>
#include <tdt_interface/msg/send_data.hpp>
#include <opencv2/opencv.hpp>
#include <vector>
#include <cmath>
#include <algorithm>
```

```
using namespace cv;
using namespace std;
using namespace rclcpp;
```

/*
整个思路:

```
ROS 2 接收校园赛实时图像
图像颜色通道分离
红色通道和蓝色通道差值
高斯模糊
二值化处理
```

```

轮廓检测
最小外接矩形拟合
筛选灯条 (这个也太难了吧 参数怎么都调不对 可恶)
灯条配对 (我超威你配对给我配对好了啊喂 为什么会给我把地面当做灯条去配对啊 )
Armor
绘制 Armor
实时显示结果
Armor
ROI
SVM 数字识别 (为什么突然给我打不开文件了啊! 可以弄了但是为什么给我把 3 识别成 6 啊喂)
目标位置
yaw / pitch
ROS 2 发布瞄准指令
*/

struct lightbar
{
    Point2f center; //储存灯条的中心点
    float width; //储存灯条的宽度
    float height; //储存灯条的高度
    float angle; //储存灯条的倾斜角度
    Point2f top; //储存灯条上边的中心点
    Point2f bottom; //储存灯条下边的中心点
    Point2f corners[4];
}; //用来储存灯条的信息

struct Armor
{
    Point2f center; //储存装甲板的中心点
    vector<Point2f> corners; //储存装甲板的四个角点
    vector<Point2f> pnpCorners; //用来储存后面计算的 pnp 的二维坐标点
    lightbar left; //储存装甲板左边的灯条
    lightbar right; //储存装甲板右边的灯条
    int number; //保存 SVM 识别出来的数字
}; //用来储存装甲板的信息

double pointdistance(Point2f p1, Point2f p2)
{
    return sqrt(
        pow(p1.x-p2.x,2)+
        pow(p1.y-p2.y,2)
    ); //计算两个点之间的距离
}

//后面创建 trackbar 先定义一些变量;

```

```

int thresholdValue = 100; //二值化阈值
int minAreaValue = 70; //灯条最小面积
int maxAreaValue = 5000; //灯条最大面积
int maxAngleValue = 30; //灯条最大允许角度
int maxCenterAngleValue = 20; //两个灯条中心连线最大允许角度
double maxDisRatioValue = 4.0; //计算两个灯条之间的距离和灯条高度的比例

// 装甲板物理尺寸 (单位: 米), 请按实测修改
const double ARMOR_WIDTH_M = 0.135; // 两灯条中心之间的水平距离 (先按模块总宽估算)
const double ARMOR_HEIGHT_M = 0.055; // 灯条上下端中点之间的距离 (图中 55mm)

class CampusNode : public rclcpp::Node
{
public:

    CampusNode()
        : Node("campus_node")
    {
        svm_ = cv::ml::SVM::load("/home/robot/campus_ws/src/campus_vision/src/digit_svm.yml");

        RCLCPP_INFO(
            this->get_logger(),
            "  校园赛节点启动! "
        ); //输出节点启动信息

        cv::namedWindow(
            "  参数",
            cv::WINDOW_NORMAL
        ); //创建参数调节窗口

        cv::createTrackbar(
            "maxDisRatio",
            "  参数",
            nullptr,
            100
        );

        cv::createTrackbar(
            "threshold",
            "  参数",
            nullptr,
            255

```

```

); //创建二值化阈值调节条

cv::createTrackbar(
    "minArea",
    " 参数",
    nullptr,
    1000
); //创建最小面积调节条

cv::createTrackbar(
    "maxArea",
    " 参数",
    nullptr,
    10000
); //创建最大面积调节条

cv::createTrackbar(
    "maxAngle",
    " 参数",
    nullptr,
    45
); //创建灯条最大角度调节条

cv::createTrackbar(
    "centerAngle",
    " 参数",
    nullptr,
    45
); //创建中心连线最大角度调节条
// 使用 nullptr 空指针来减小算法压力 不然会卡死程序 不会弹出窗口

cv::setTrackbarPos(
    "threshold",
    " 参数",
    thresholdValue
); //设置二值化阈值初始值

cv::setTrackbarPos(
    "minArea",
    " 参数",
    minAreaValue
); //创建最小面积调节条

```

```

cv::setTrackbarPos(
    "maxArea",
    " 参数",
    maxAreaValue
); //创建最大面积调节条

cv::setTrackbarPos(
    "maxAngle",
    " 参数",
    maxAngleValue
); //创建最大角度调节条

cv::setTrackbarPos(
    "centerAngle",
    " 参数",
    maxCenterAngleValue
); //创建中线连线最大角度调节条

cv::setTrackbarPos(
    "maxDisRatio",
    " 参数",
    static_cast<int>(maxDisRatioValue * 10) //还必须要有空格?
); //创建灯条之间的距离和灯条高度比例的调节条

image_sub_ =
    this->create_subscription<sensor_msgs::msg::CompressedImage>(
        "/camera/image_raw/compressed",
        10,
        std::bind(
            &CampusNode::imageCallback,
            this,
            std::placeholders::_1
        )
    ); //创建图像订阅器，订阅校园赛发送的压缩图像

frame_sub_ =
    this->create_subscription<tdt_interface::msg::AutoAimFrame>(
        "/autoaim/frame",
        1,
        std::bind(
            &CampusNode::frameCallback,
            this,
            std::placeholders::_1
        )
    ); //创建 AutoAimFrame 订阅器，获取 camera_info

```

```

send_data_pub_=
    this->create_publisher<tdt_interface::msg::SendData>(
        "/target_angles_player_1",
        10
    );//创建瞄准指令发布者
}

```

```

void processFrame(cv::Mat& frame)
{
    Mat result=frame.clone();//复制一张图像用来绘制识别结果
    static int frameCount=0;//记录当前处理的帧数

    frameCount++;//每收到一帧图像就加 1

    if(frameCount%3!=0)
    {
        return;//每 3 帧处理 1 帧,降低 CPU 占用
    }

    maxDisRatioValue=
        cv::getTrackbarPos(
            "maxDisRatio",
            " 参数"
        )/10.0;//获取当前灯条最大距离比例

    thresholdValue=
        cv::getTrackbarPos(
            "threshold",
            " 参数"
        );

    minAreaValue=
        cv::getTrackbarPos(
            "minArea",
            " 参数"
        );

    maxAreaValue=
        cv::getTrackbarPos(
            "maxArea",
            " 参数"
        );
}

```

```

maxAngleValue=
cv::getTrackbarPos(
    "maxAngle",
    " 参数"
);

//图像通道分离

vector<Mat>channels;//用来储存 B、G、R 三个通道

cv::split(
    frame,
    channels
);//将图像分离成 B、G、R 三个通道

//红色通道和蓝色通道差值

Mat red_diff;//用来储存红色差值图

cv::subtract(
    channels[2],
    channels[0],
    red_diff
);//计算红色通道和蓝色通道的差值

cv::GaussianBlur(
    red_diff,
    red_diff,
    Size(9,9),
    0
);//对差值图进行高斯模糊处理

//二值化

Mat thre;//用来储存二值化后的图像

```

```

cv::threshold(
    red_diff,
    thre,
    thresholdValue,
    255,
    THRESH_BINARY
); //对差值图进行二值化处理

//轮廓检测

vector<vector<Point>>>contours; //用来储存轮廓的点集

cv::findContours(
    thre,
    contours,
    RETR_EXTERNAL,
    CHAIN_APPROX_SIMPLE
); //寻找轮廓

vector<lightbar>lightbars; //用来储存所有符合条件的灯条
vector<Armor>armors; //储存识别到的所有装甲板

//筛选灯条

for(size_t i=0;
    i<contours.size();
    i++)
{
    double area=
        contourArea(
            contours[i]
        ); //计算的是轮廓的面积

    RotatedRect rect=
        minAreaRect(
            contours[i]
        ); //计算的是轮廓的最小外接矩形

```

```

float height=
    rect.size.height; //获取轮廓的高度

float width=
    rect.size.width; //获取轮廓的宽度

if(width>height)
{
    swap(
        width,
        height
    ); //如果宽度大于高度，则交换宽度和高度，因为灯条的高度应该大于宽度
}

Point2f pts[4]; //用来储存轮廓的四个角点

rect.points(pts); //获取轮廓的四个角点

double edgeLength[4]; //用来储存四条边的长度

for(int k=0;k<4;k++)
{
    edgeLength[k]=
        pointdistance(
            pts[k],
            pts[(k+1)%4]
        ); //计算当前边的长度
}

int shortEdge=0; //记录最短边的编号

for(int k=1;k<4;k++)
{
    if(edgeLength[k]<edgeLength[shortEdge])
    {
        shortEdge=k; //找到最短的那条边
    }
}

int oppositeEdge=
    (shortEdge+2)%4; //找到与最短边相对的另一条短边

```

```

Point2f point1=
(
    pts[shortEdge]+
    pts[(shortEdge+1)%4]
)/2; //计算第一条短边的中心点

Point2f point2=
(
    pts[oppositeEdge]+
    pts[(oppositeEdge+1)%4]
)/2; //计算另一条短边的中心点

Point2f top;
Point2f bottom;

if(point1.y<point2.y)
{
    top=point1;
    bottom=point2;
} //根据两个短边中心点的 y 坐标判断上下关系
else
{
    top=point2;
    bottom=point1;
} //根据两个短边中心点的 y 坐标判断上下关系

double dx=
    bottom.x-top.x; //计算灯条在 x 方向上的变化

double dy=
    bottom.y-top.y; //计算灯条在 y 方向上的变化

lightbar lb; //用来储存当前灯条的信息

```

```

lb.center=
    rect.center; //储存灯条的中心点

lb.width=
    width; //储存灯条的宽度

lb.height=
    height; //储存灯条的高度

lb.top=
    top; //储存灯条上边的中心点

lb.bottom=
    bottom; //储存灯条下边的中心点

lb.angle=
    atan2(dx,dy)*180/CV_PI; //计算灯条相对于竖直方向的倾斜角度
for(int i=0;i<4;i++){
    lb.corners[i]=pts[i];
}

if(
    height/width>1.5 &&
    area> minAreaValue &&
    area<maxAreaValue &&
    abs(lb.angle)<maxAngleValue
)
{
    lightbars.push_back(
        lb
    ); //将符合条件的灯条加入灯条数组
    static int lb_count=0;
    if(++lb_count%50==0)
        RCLCPP_INFO(this->get_logger(), " 灯条总数 %d", lb_count);

    for(int k=0;k<4;k++){
        {
            cv::circle(
                result,
                pts[k],
                4,
                Scalar(0,255,0),
                -1
            );
        }
    }
}

```

```

    } //绘制灯条的四个角点

    vector<Point>lightbarCorners; //用来储存灯条的四个角点

    for(int k=0;k<4;k++)
    {
        lightbarCorners.push_back(
            Point(
                cvRound(pts[k].x),
                cvRound(pts[k].y)
            )
        );
    } //将灯条的四个角点转换成整数坐标

    cv::polylines(
        result,
        lightbarCorners,
        true,
        Scalar(255,0,0),
        2
    ); //绘制灯条的轮廓

    cv::circle(
        result,
        top,
        5,
        Scalar(0,255,0),
        -1
    ); //绘制灯条上边的中心点

    cv::circle(
        result,
        bottom,
        5,
        Scalar(255,0,0),
        -1
    ); //绘制灯条下边的中心点
    }
}

```

```

//灯条配对

for(size_t i=0;
    i<lightbars.size();
    i++)
{
    for(size_t j=i+1;
        j<lightbars.size();
        j++)
    {
        double dis=
            pointdistance(
                lightbars[i].center,
                lightbars[j].center
            );//计算两个灯条中心点之间的距离

        double heightDiff=
            abs(
                lightbars[i].center.y-
                lightbars[j].center.y
            );//计算两个灯条之间的高度差

        double angleDiff=
            abs(
                lightbars[i].angle-
                lightbars[j].angle
            );//计算两个灯条之间的角度差

        double centerDx =
            lightbars[j].center.x-
            lightbars[i].center.x;//计算两个灯条中心点在 x 方向上的距离

        double centerDy =
            lightbars[j].center.y-
            lightbars[i].center.y;//计算两个灯条中心点在 y 方向上的距离

        double centerAngle=
            atan2(
                centerDy,
                centerDx
            );
    }
}

```

```

        )*180/CV_PI; //计算两个灯条中心连线相对于水平方向的角度
double avgHeight=
    (
        lightbars[i].height+
        lightbars[j].height
    )/2.0; //计算两个灯条的平均高度

double disRatio=
    dis/avgHeight;
    //计算两个灯条之间的距离和灯条高度的比例

double heightRatio=
    max(
        lightbars[i].height,
        lightbars[j].height
    )/
    min(
        lightbars[i].height,
        lightbars[j].height
    ); //计算两个灯条高度之间的比例

if(

    disRatio>1.5 &&
    disRatio<maxDisRatioValue &&
    heightDiff<avgHeight*0.5 &&
    angleDiff<15 &&
    heightRatio<1.5 &&
    abs(centerAngle)<maxCenterAngleValue

)
{
    Armor armor; //用来储存装甲板的信息
    armor.number=-1; //认为还没有结果 先初始化

    if(
        lightbars[i].center.x<
        lightbars[j].center.x
    )
    {
        armor.left=
            lightbars[i];
    }
}

```

```

        armor.right=
            lightbars[j];
    }
    else
    {
        armor.left=
            lightbars[j];

        armor.right=
            lightbars[i];
    } //判断两个灯条的左右关系

armor.center=
    (
        armor.left.center+
        armor.right.center
    )/2; //计算装甲板的中心点

armor.corners.push_back(
    armor.left.top
); //将左灯条上边中心点存入装甲板角点

armor.corners.push_back(
    armor.right.top
); //将右灯条上边中心点存入装甲板角点

armor.corners.push_back(
    armor.right.bottom
); //将右灯条下边中心点存入装甲板角点

armor.corners.push_back(
    armor.left.bottom
); //将左灯条下边中心点存入装甲板角点

armor.pnpCorners = armor.corners; //暂时将角点复制给 pnpCorners

// ----- PnP 解算目标位置 -----
// 仅当相机内参已获取 (frameCallback 已写入) 时才做
if(!cameraMatrix_.empty() && !distCoeffs_.empty())
{
    static int pnp_count=0;
    if(++pnp_count%30==0)
        RCLCPP_INFO(this->get_logger(), " 进入 PnP 段 %d 次", pnp_count);
    // 四个角点在装甲板平面上的真实 3D 坐标, z=0 (装甲板所在平面)

```

```

// 对应顺序: left.top, right.top, right.bottom, left.bottom
vector<Point3f> objectPoints = {
    Point3f(-ARMOR_WIDTH_M/2.0f, -ARMOR_HEIGHT_M/2.0f, 0.0f),
    Point3f( ARMOR_WIDTH_M/2.0f, -ARMOR_HEIGHT_M/2.0f, 0.0f),
    Point3f( ARMOR_WIDTH_M/2.0f,  ARMOR_HEIGHT_M/2.0f, 0.0f),
    Point3f(-ARMOR_WIDTH_M/2.0f,  ARMOR_HEIGHT_M/2.0f, 0.0f)
};

vector<Point2f> imagePoints;
for(size_t k=0;k<armor.pnpCorners.size();k++)
{
    imagePoints.push_back(armor.pnpCorners[k]); // 与上面 3D 点一一对应的
}

Mat rvec, tvec; // 旋转向量、平移向量 (平移即目标在相机坐标系下的位置)
bool ok = cv::solvePnP(
    objectPoints,
    imagePoints,
    cameraMatrix_,
    distCoeffs_,
    rvec,
    tvec,
    false,
    cv::SOLVEPNP_IPPE // 对平面目标更稳的解算方法
);

if(ok)
{
    static int ok_count=0;
    if(++ok_count%30==0)
        RCLCPP_INFO(this->get_logger(), "solvePnP 成功 %d 次, tz=%.3f", ok_count, tvec.at<double>(2));
    double tx = tvec.at<double>(0); // 目标在相机系下 X (右为正)
    double ty = tvec.at<double>(1); // 目标在相机系下 Y (下为正)
    double tz = tvec.at<double>(2); // 目标在相机系下 Z (前为正, 即距离)

    if(tz > 0)
    {
        double yaw_deg = atan2(tx, tz) * 180.0 / CV_PI; // 目标在相机系下的偏角
        double pitch_deg = atan2(ty, tz) * 180.0 / CV_PI; // 目标在相机系下的俯角

        // 转成世界光轴角: 世界 yaw = 自身当前世界 yaw + 目标在相机系下的偏角
        double world_yaw = last_state_.yaw_degrees + yaw_deg;

        tdt_interface::msg::SendData send_msg;
        send_msg.yaw = static_cast<float>(world_yaw);
        send_msg.pitch = static_cast<float>(pitch_deg);
    }
}

```

```

        send_msg.if_shoot = true; // 允许开火
        send_data_pub_ -> publish(send_msg); // 发布瞄准指令

        // 在画面上显示解算的 yaw/pitch 和距离
        string info = "yaw:" + to_string(yaw_deg).substr(0,6) +
            " pitch:" + to_string(pitch_deg).substr(0,6) +
            " dist:" + to_string(tz).substr(0,6);
        putText(result, info, Point(10, 30),
            FONT_HERSHEY_SIMPLEX, 0.7, Scalar(0,255,255), 2);
    }
}
// ----- PnP 解算结束 -----

vector<Point> drawCorners; // 用来储存装甲板的四个角点

for(size_t k=0; k<armor.corners.size(); k++){
    const Point2f& p=armor.corners[k];
    drawCorners.emplace_back(
        cvRound(p.x),
        cvRound(p.y)
    ); // 将角点坐标转化为整数
    putText(
        result,
        to_string(k),
        Point(cvRound(p.x)+5, cvRound(p.y)-5),
        FONT_HERSHEY_COMPLEX,
        0.8,
        Scalar(110,200,25),
        2
    ); // 在每个角点旁边写上编号
}

Rect numberRect=boundingRect(drawCorners); // 计算装甲板角点的外接矩形
int expandX = numberRect.width*0.2; // 将水平宽度扩大原宽度的 20%
int expandY = numberRect.height*0.2; // 将垂直高度扩大原高度的 20%

numberRect.x -= expandX; // 左边向左扩展
numberRect.y -= expandY; // 上边向上扩展

numberRect.width += 2* expandX; // 左右两边一起扩展
numberRect.height += 2* expandY; // 上下两边一起扩展

Rect imageRect(0,0,frame.cols,frame.rows); // 划一块区域 和图像一样大小

```

```

numberRect=numberRect&imageRect; //取交集 防止越界
Mat numberROI = frame(numberRect); //从画面中截取感兴趣区域
if(numberROI.empty()){
    continue;
}
Mat gray;
cvtColor(numberROI,
        gray,
        COLOR_BGR2GRAY);
resize(gray,
        gray,
        Size(20,28));
Mat feature;
gray.convertTo(feature,
                CV_32F,
                1.0/255.0);
feature=feature.reshape(1,1);
int prediction = static_cast<int>(svm_->predict(feature)); //获取 SVM 识别结果
armor.number=prediction; //将识别到的数字存入 armor.number
armors.push_back(armor); //将当前识别到的加入装甲板组
static int am_count=0;
if(++am_count%10==0)
    RCLCPP_INFO(this->get_logger(), " 装甲板总数 %d", am_count);
string text = to_string(armor.number); //将数字转换为 string 字符串
putText(result,
        text,
        Point(numberRect.x,numberRect.y),
        FONT_HERSHEY_COMPLEX,
        1,
        Scalar(255,255,255),
        2);

cv::polylines(
    result,
    drawCorners,
    true,
    Scalar(0,255,0),
    2
); //绘制装甲板的轮廓

cv::circle(
    result,
    armor.center,
    5,
    Scalar(255,0,0),

```

```

        -1
    ); //绘制装甲板的中心点
    }
}

cv::imshow(
    "result",
    result
); //显示最终识别结果

cv::Mat parameterImage =
    cv::Mat::zeros(
        150,
        600,
        CV_8UC3
    ); //创建一张黑色参数窗口背景

cv::imshow(
    " 参数",
    parameterImage
); //显示参数窗口

cv::waitKey(1); //等待 1 毫秒, 让 OpenCV 窗口刷新
}

```

private:

```

rclcpp::Subscription<
    sensor_msgs::msg::CompressedImage
>::SharedPtr image_sub_; //用来储存图像订阅器

cv::Ptr<cv::ml::SVM> svm_; //用来储存 SVM 模型

cv::Mat cameraMatrix_; //储存相机内参矩阵 K
cv::Mat distCoeffs_; //储存相机畸变参数 D

rclcpp::Subscription<
    tdt_interface::msg::AutoAimFrame
>::SharedPtr frame_sub_; //储存 AutoAimFrame 订阅器

rclcpp::Publisher<
    tdt_interface::msg::SendData

```

```

>::SharedPtr send_data_pub_; //储存瞄准指令发布者

tdt_interface::msg::AutoAimState last_state_; //保存最近一次 AutoAimFrame 里的自身状态

// frameCallback: 从 AutoAimFrame 消息中取出 camera_info, 保存到 cameraMatrix_/distCoeffs_, 供
void frameCallback(
    const tdt_interface::msg::AutoAimFrame::SharedPtr msg
)
{
    // CameraInfo.k 是按行存储的 3x3 内参矩阵 [fx, 0, cx, 0, fy, cy, 0, 0, 1]
    if(msg->camera_info.k.size() >= 9)
    {
        cameraMatrix_ = (cv::Mat_<double>(3,3) <<
            msg->camera_info.k[0], msg->camera_info.k[1], msg->camera_info.k[2],
            msg->camera_info.k[3], msg->camera_info.k[4], msg->camera_info.k[5],
            msg->camera_info.k[6], msg->camera_info.k[7], msg->camera_info.k[8]);
    }
    // distortion_model 里畸变系数一般为 5 个 [k1,k2,p1,p2,k3]
    if(!msg->camera_info.d.empty())
    {
        distCoeffs_ = cv::Mat(msg->camera_info.d).clone();
    }
    // 保存当前云台世界姿态, 供把相机系 yaw 换成世界 yaw
    last_state_ = msg->state;
    static int fc_count=0;
    if(++fc_count%100==0)
        RCLCPP_INFO(this->get_logger(), "frameCallback 收到 %d 次, cameraMatrix_ empty=%d", fc_count, cameraMatrix_.empty());
}

void imageCallback(
    const sensor_msgs::msg::CompressedImage::SharedPtr msg
)
{
    cv::Mat frame=
        cv::imdecode(
            msg->data,
            cv::IMREAD_COLOR
        ); //将 ROS 2 中的压缩图像解码成 OpenCV 图像

    if(frame.empty())
    {
        RCLCPP_WARN(
            this->get_logger(),
            " 图像解码失败"
        ); //输出图像解码失败的警告
    }
}

```

```

        return;
    }

    processFrame(
        frame
    );//将接收到的图像交给 OpenCV 处理
    static int ic_count=0;
    if(++ic_count%100==0)
        RCLCPP_INFO(this->get_logger(), "imageCallback 收到 %d 帧", ic_count);
}

};

int main(int argc, char* argv[])
{
    rclcpp::init(
        argc,
        argv
    );//初始化 ROS 2

    auto node=
        std::make_shared<CampusNode>();//创建校园赛节点

    rclcpp::spin(
        node
    );//让节点持续运行并等待 ROS 2 消息

    rclcpp::shutdown();//关闭 ROS 2

    return 0;
}

```